

Matrix Multiplication Benchmark

Individual Assignment 1 — Report

Carlos Montesdeoca Vega

repository: <https://github.com/CarlosMontesdeoca21/BasicMatrixMultiplication>

October 3, 2026

Abstract

This report investigates the performance of a basic dense matrix multiplication algorithm across three programming languages: Python, Java, and C. By employing a direct triple-loop implementation, this study aims to observe how matrix size impacts execution time and memory usage under documented execution conditions. The methodology incorporates best practices such as the separation of production code from benchmark scripts, parameterization of matrix sizes, and multiple runs per experiment to ensure robust statistical data. The evidence collected focusing on median execution times, interquartile range (IQR) variability, and memory metrics is analyzed against the theoretical $O(n^3)$ complexity model to understand the distinct overheads introduced by compiled, JIT-compiled, and interpreted execution paradigms.

1 Introduction

The goal of this work is to refactor, organize, and benchmark a basic dense matrix multiplier, comparing performance across multiple implementations (Python, Java, and C) while applying software engineering best practices:

- Clean separation of production code from test and benchmark scripts.
- Parameterization of matrix sizes ($n \in \{10, 100, 250\}$).
- Multiple runs per experiment combined with warm-up phases to obtain reliable statistics and mitigate system noise.
- Integration of memory tracking and professional benchmarking tools.

2 Code and organization

The project repository includes a structured hierarchy with implementations under the `Activity1/code` folder (Python, Java, and C) alongside root management scripts:

- **Python Implementation & Automation Scripts:** Includes core matrix creation and multiplication modules, individual benchmark runners featuring explicit warm-up phases, a correctness validator based on checksum verification, an input generator

script (`generate_inputs.py`), a global result unifier (`merge_results.py`), and an automated plotting script (`plot_results.py`) that computes IQR variability and exports performance charts.

- **Java and C Implementations:** Feature dedicated source files and benchmark runners (also integrating warm-up iterations) compiled with specific optimization flags and linking process APIs like `-lpsapi` for precise memory tracking in C.
- **Code Separation:** Production logic is strictly isolated from testing, validation, and benchmarking execution scripts.
- **Parameterization:** Tested matrix sizes are fully parameterized, targeting the sequence $n \in \{10, 100, 250\}$.

3 Experimental methodology

- **Measurement:** High-resolution wall-clock timing calls were utilized across all scripts (e.g., `perf_counter` in Python, high-resolution timers in Java and C). Memory consumption was tracked using implementation-specific tools (the Windows process API `psapi` for C and the `Runtime` class for Java).
- **Universal Warm-up Phase & Repetitions:** A fixed warm-up phase (2 rounds) was explicitly executed prior to recording official metrics across all three languages. The technical justification varies by programming paradigm:
 - *Java:* Essential for the Java Virtual Machine (JVM) and its Just-In-Time (JIT) compiler, allowing HotSpot to dynamically optimize bytecode into native machine code.
 - *C:* Mitigates cold-start anomalies, ensuring that CPU instruction and data caches and page tables are adequately populated before official timing commences.
 - *Python:* Stabilizes interpreter overhead, module lookup tables, and internal dynamic runtime states.

Following the warm-up, 5 official consecutive repetitions were executed per matrix size to capture statistical variance.

- **Correctness Validation:** Correctness was rigorously validated by computing a checksum (sum of all resulting matrix elements) and ensuring the numerical values matched across all three language implementations for the baseline size of n .
- **Recorded Results:** Data artifacts are consolidated in the project’s data folder, feeding automated performance and memory footprint plotting scripts.

4 Data and statistics

The data extracted from the consolidated benchmarking CSVs are summarized below. For each language-size configuration ($n \in \{10, 100, 250\}$), the median execution time, the interquartile range (IQR) variability, and the median memory consumption are reported to ensure robust statistical evaluation.

4.1 Numerical summary

Table 1: Benchmark results (Median time, IQR variability, and Median memory) by language and matrix size.

Language	Size (n)	Median Time (s)	IQR Time (s)	Median Memory (KB)
C	10	0.000000	0.000000	3816.0
C	100	0.001000	0.000000	4096.0
C	250	0.012000	0.002000	5480.0
Java	10	0.000051	0.000015	1068.0
Java	100	0.003441	0.000452	1309.0
Java	250	0.014449	0.000346	2549.0
Python	10	0.000116	0.000005	27140.0
Python	100	0.098353	0.008733	28364.0
Python	250	1.647374	1.688286	34340.0

Note on data aggregation and trends: The values reported above correspond to the median times and memory footprints extracted across multiple official repetitions, effectively mitigating the influence of isolated system outliers. A notable behavioral pattern observed during execution is that Python exhibits a progressive increase in execution time across successive repetitions for larger matrices (ej., $n = 250$), driven by dynamic interpreter overhead and garbage collection cycles, whereas compiled languages like C and Java maintain highly stable execution times across all runs. This rebound over time can be observed in the CSV files attached to the GitHub repository in the results folder.

4.2 Graphical analysis and plots

To visually evaluate the scaling behavior and resource utilization of the different implementations, comparative performance and memory footprint plots were automatically generated from the benchmark data.

4.2.1 Execution Time Analysis

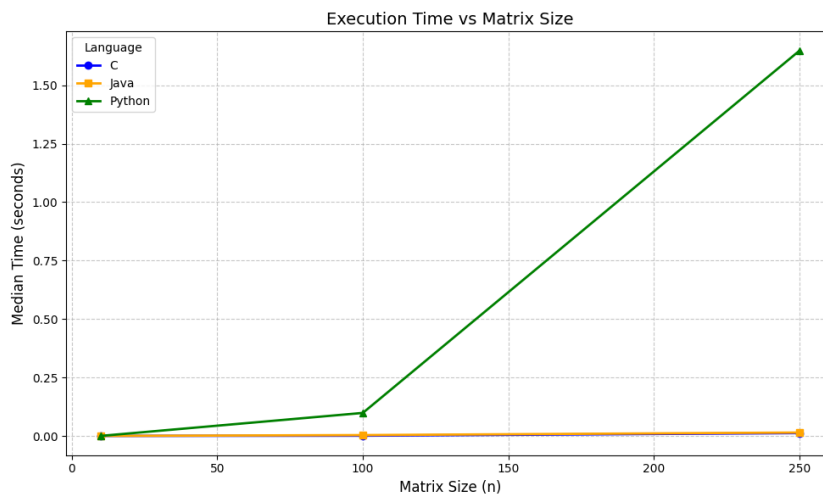


Figure 1: Execution time scaling across matrix sizes for C, Java, and Python.

Analysis: As observed in Figure 4.2.1, execution time grows non-linearly, following the theoretical $O(n^3)$ computational complexity model. However, the magnitude of the slope differs dramatically: native C and JIT-optimized Java maintain extremely flat growth rates with negligible times at $n = 10$ and $n = 100$. In sharp contrast, Python exhibits a steep upward inflection as the matrix size reaches $n = 250$, reflecting the heavy overhead of interpreted loops and dynamic type checking.

4.2.2 Memory Consumption Analysis

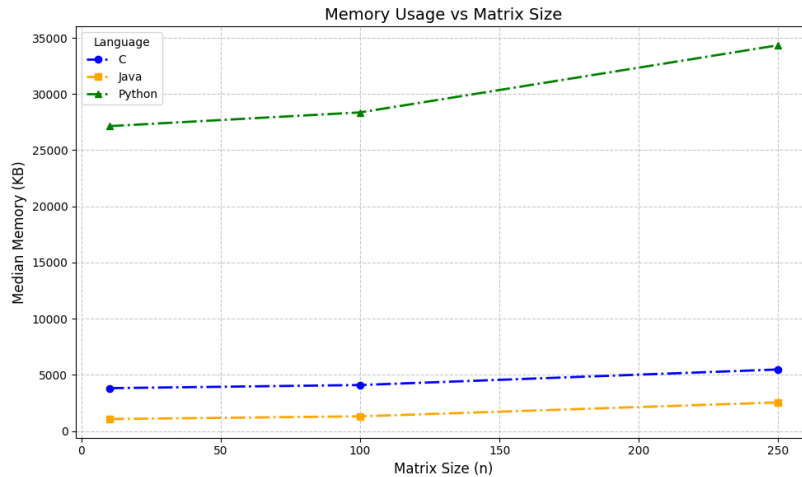


Figure 2: Memory consumption footprint across matrix sizes for all evaluated languages.

Analysis: As presented in Figure 4.2.2, Python demonstrates a significantly higher baseline memory footprint (starting around 27 MB) due to its object-oriented dynamic structure, interpreter overhead, and reference counting mechanisms. Conversely, C and Java maintain highly optimized, compact memory footprints (ranging from a few kilobytes to a few megabytes), scaling much more efficiently as the input matrices grow in dimension.

5 Analysis and discussion

5.1 Observed performance comparison

The experimental results align closely with the architectural characteristics of each execution environment:

- **C Implementation:** Consistently exhibits the fastest execution times across all tested matrix sizes. This superior performance stems from native compilation, aggressive compiler optimization flags (e.g., `-O3`), and flat, contiguous memory layout that maximizes CPU cache locality.
- **Java Implementation:** Performs competitively at small and medium scales. Although initial execution encounters interpretation overhead, the Java Virtual Machine (JVM) leverages its Just-In-Time (JIT) compiler and HotSpot optimization to compile hot loops into efficient native machine code.
- **Python Implementation:** Demonstrates a sharp escalation in execution time as n scales to 250. This behavior is directly attributable to CPython’s dynamic

interpreter overhead, reference counting, and the absence of native type optimization inside the explicit triple loop.

5.2 Measurement reliability and environment factors

The methodology incorporated 5 official repetitions preceded by explicit warm-up phases to minimize cold-start anomalies and transient system noise. Furthermore, utilizing the median alongside the interquartile range (IQR) guarantees robustness against isolated outliers caused by background operating system tasks. However, exact execution times remain strictly bounded by underlying hardware specifications, CPU clock throttling, and background load.

6 Benchmarking and profiling tools

- **Used in the project:**

- *Timing & Metrics:* High-resolution wall-clock timers across all languages, alongside implementation-specific tools like the Windows process API (`psapi`) in C and the `Runtime` class in Java for memory tracking.
- *Data Processing & Visualization:* Python scripts utilizing `pandas` for data unification and `matplotlib` for automated plot generation and IQR variability computation.

- **Recommended for advanced analysis:**

- *For C:* Linux `perf`, `gprof`, or Callgrind with KCacheGrind for detailed instruction-level profiling and cache-miss analysis.
- *For Java:* Java Flight Recorder (JFR), VisualVM, and specialized micro-benchmarking suites to handle JVM warm-up phases and thread synchronization accurately.
- *For Python:* `cProfile` for function-level call trees, `pyinstrument` or `scalene` for granular CPU and memory profiling, and `timeit` for precise micro-benchmarks.

7 Optimization recommendations

To overcome the performance bottlenecks identified during benchmarking, several optimization strategies can be implemented across algorithmic, language-specific, and parallel dimensions:

- **Algorithmic Optimization:** Implement blocked (tiled) matrix multiplication to significantly improve CPU cache hit rates (L1/L2/L3) and reduce memory bandwidth pressure for large matrices.
- **Language-Specific Enhancements:**
 - *C:* Enable aggressive compiler optimization flags (ej., `-O3`) and hardware-specific vectorization flags (ej., `-march=native`).

- *Java*: Tune Java Virtual Machine (JVM) parameters, heap sizes, and ensure thorough warm-up routines to maximize JIT compilation efficiency.
- *Python*: Avoid pure nested loops by utilizing highly optimized vector/matrix libraries (such as NumPy backed by BLAS or OpenBLAS), or accelerate execution using JIT compilers like Numba or C extensions via Cython.
- **Parallelism & Concurrency**: Exploit multi-core hardware architectures by integrating OpenMP directives in C or parallel streams and multi-threading in Java.
- **Robust Benchmarking Practices**: Scale automated repetitions to 15–30 runs, incorporate outlier filtering mechanisms, and systematically record comprehensive hardware metadata.

8 Conclusions

- **Performance Findings**: Within the evaluated scope ($n \in \{10, 100, 250\}$), C significantly outperforms Java, and both drastically outperform Python when utilizing a naive triple-loop implementation.
- **Bottlenecks & Optimization**: The primary computational bottleneck resides in the core multiplication loops. Applying algorithmic tiling, leveraging native compilation flags, or switching to mathematically accelerated libraries (such as BLAS/NumPy) is essential to reduce execution overhead.
- **Experimental Rigor**: Ensuring reproducible benchmarks requires strict separation of production code, universal warm-up phases to stabilize runtimes (JIT/caches), multiple official repetitions, and robust statistical analysis via medians and interquartile ranges (IQR).
- **AI Collaboration Disclosure**: In accordance with transparent academic practices, **Gemini** was utilized during the development of this project. Its purpose was strictly limited to assisting with technical writing, structuring the academic report in LaTeX, debugging compilation errors, and refining the data analysis methodology. All benchmarking executions, code implementations, and final validations were independently performed and verified by the author.